

GIT

Installing git

- Sudo su - root
- Yum install git
- Git --version

Tasks:

1)pushing file to remote repo [github]

Flow: have to create a file in working copy where we initialize git that will be in untracked status that is added to staging area [a temporary place in .git folder] and committed to local repository [a place where log creates] and after adding index name of remote url the content can be pushed to the remote repository

Procedure

1. have to create a directory
2. git init
3. git config -global user.name "abdulrahman01"
4. Git config -global user.email "abdul96@gamil.com"
5. Git config -l
6. Git add [file name] or git add * → to add all files
7. Git commit -m "msg"
8. Git status -s
9. Git show - to show changes in the commit
10. Git log --oneline
11. Create a github repo and copy the url
12. Git remote add abd[indexname] master[branchname]
13. Git push [indexname] [branchname]

14. `Git branch [feature branch name]`
15. `Git merge [opposite branch name]` usually we will be standing on master branch and have to give git merge opposite branch name so that the content is copied from feature branch to master branch
16. `Git checkout [branch name]` to change to that given branch name.
17. `Git config -global merge.tool vimdiff`
18. `Git config -global merge.conflict style dif3`
19. `Git config -global merge.prompt false`

- If you add some content in git added file then it goes to untracked file then again you've add and commit it
- Head is a pointer
- If you didn't commit until that master branch would not create after giving the 1st commit only master branch will gets created
- We have to take back up which is branching to avoid loss of data, to prevent the master content, to prevent the last modification time stamp in master
- Before git merge have to check git log ,are we residing at the right branch.

Task2:

Roll back from staging area to working copy

Ans:

Git rm -cached [filename]

```
git ls-files -s
```

```
[root@ip-172-31-39-124 git]# git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   test2

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    test3

[root@ip-172-31-39-124 git]# git rm --cached test2
rm 'test2'
[root@ip-172-31-39-124 git]# git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    test2
    test3

nothing added to commit but untracked files present (use "git add" to track)
[root@ip-172-31-39-124 git]#
```

2nd Assignment

Rollback the content from local repository to staging area

Having 3 files test1,test2,test3

committed test1 and test2

added changes in test2 and committed the 3rd file along with test2 changes ,if I give **git reset -soft** with 2nd commit ID then the new file and test2 file changes rollbacked from local repo to staging area and the changes in the file persists.

```
[root@ip-172-31-39-124 git1]# git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   test2
        new file:   test3

[root@ip-172-31-39-124 git1]# git commit -m "recommit for test2 and test3"
[master 3ecfe95] recommit for test2 and test3
 2 files changed, 4 insertions(+)
 create mode 100644 test3
[root@ip-172-31-39-124 git1]# git status -s
[root@ip-172-31-39-124 git1]# git log --oneline
3ecfe95 (HEAD -> master) recommit for test2 and test3
b70e4fc 2n commit test2 file with content 1,2
f8847ca 1st commit conten=1
[root@ip-172-31-39-124 git1]# git reset --soft
[root@ip-172-31-39-124 git1]# git reset --soft b70e4fc
[root@ip-172-31-39-124 git1]# git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   test2
        new file:   test3
```

Assignment 3

Roll back from local repository to working copy

Having 3 files test1,test2,test3

And added some change in test2 and 3 if you give git reset -mixed with previous commit id then the changes would come back from local repository to working copy [unstaged]

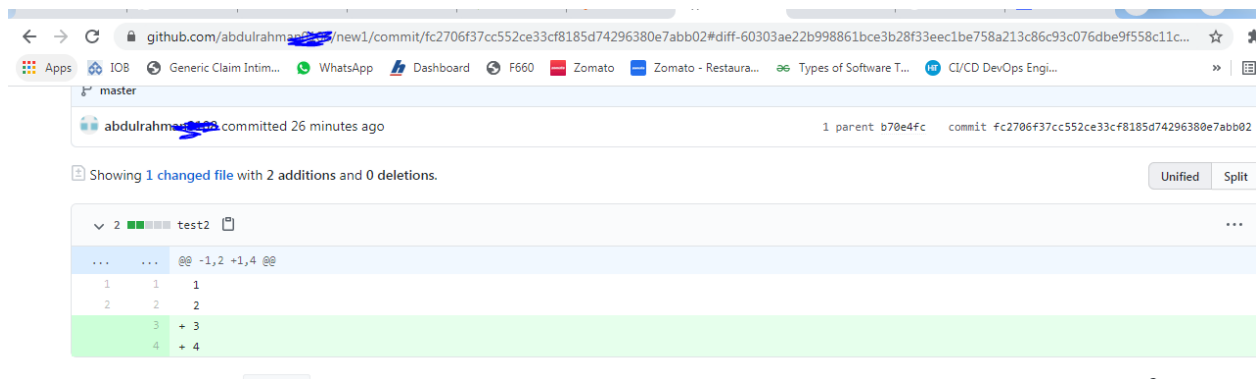
```
[root@ip-172-31-39-124 git1]# git commit -m "4th commit recommit"
[master ee37249] 4th commit recommit
 2 files changed, 4 insertions(+)
 create mode 100644 test3
[root@ip-172-31-39-124 git1]# git status -s
[root@ip-172-31-39-124 git1]# git status
On branch master
nothing to commit, working tree clean
[root@ip-172-31-39-124 git1]# git log --oneline
ee37249 (HEAD -> master) 4th commit recommit
b70e4fc 2n commit test2 file with content 1,2
f8847ca 1st commit conten=1
[root@ip-172-31-39-124 git1]# git reset --mixed b70e4fc
Unstaged changes after reset:
M      test2
[root@ip-172-31-39-124 git1]# git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test2

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test3
```

Assignment 4

Rolling back from github to local repo

- Having 3 files t1,t2,t3 ,
- T1 committed and [1st commit]
- t2 gets committed [2nd commit] separately and pushed to github
- Now added some changes to t2 gets committed [3rd commit] and pushed to github
- Now if I git revert with the 2nd commit id and again give git push
- It will roll back to the 2nd commit with a new commit id and if you refresh the github page you can notice the changes of the 3rd commit gets deleted also the change will be reflected in the t2 files also the content will be no longer in the file.



After git revert in below

Revert "3rd commit test2 content=4 wrong commit"

This reverts commit [fc2706f](#).

 master

 [abdulrahman0208](#) committed 26 minutes ago

 Showing **1 changed file** with **0 additions** and **2 deletions**.

▼ 2  test2 

... ... @@ -1,4 +1,2 @@

1 1 1

2 2 2

3 - 3

4 - 4

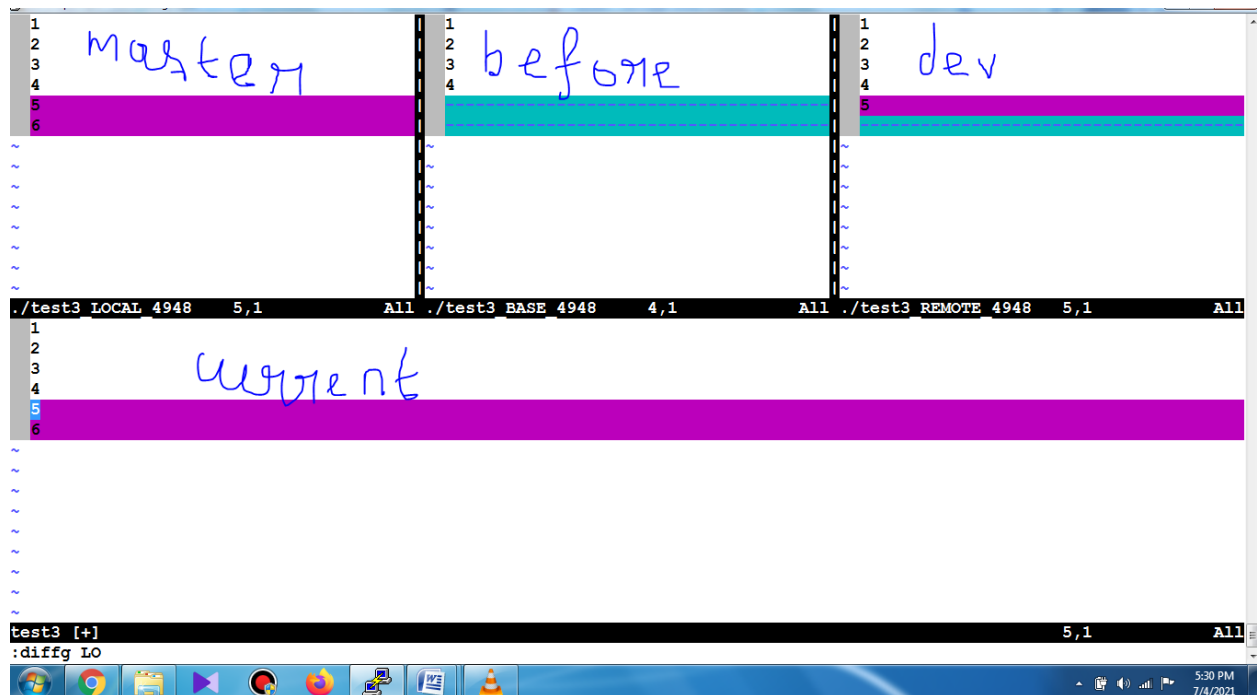
2nd scenario merge conflict

Make 2 commits in master and take a feature branch, make a 3rd commit in feature branch, checkout to master make 3rd commit in master, give git merge from master,

- Git config -global merge.tool vimdiff
- Git config -global merge.conflict style diff3
- Git config -global mergetool.prompt false
- Git mergetool
- Ctrl + shift +w
- Esc to revert the moving to re,lo,ba

- :diffg RE,LO,BA

After giving the above command the down portion has to be made as desired



Also it will create a backup file

```

cat /dev
W-r--r-- 1 root root 2 Jul 4 11:04 test1
W-r--r-- 1 root root 6 Jul 4 11:28 test2
W-r--r-- 1 root root 12 Jul 4 12:01 test3
W-r--r-- 1 root root 45 Jul 4 11:30 test3.orig

```

Assignment 6

Deleting a commit log in git

- Git rebase -i [commit id]
 - Edit pick to drop

```
drop beb26a6 3rd commit from dev content 5 in test3
```

```
# Rebase 7d4a83a..b66e5d1 onto 7d4a83a (1 command)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .      create a merge commit using the original merge commit's
# .      message (or the oneline, if no original merge commit was
# .      specified); use -c <commit> to reword the commit message
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
~
~
:wq!
```

```
[root@ip-172-31-39-124 git2]# git log --oneline
b66e5d1 (HEAD -> master) 6th commit after git merge tool corrected content but its not in added state
7d4a83a 5th commit from master test3 with 5,6
5eb28a5 (dev) 3rd commit from dev content 5 in test3
ddccede Merge branch 'dev' changed the content of t2 file eventhough having 2 commits from dev and made a 2nd commit in master to perfrom a git conflict
9555f23 2nd commit from master test2 content 3
36f34a0 2nd commit from dev content 12,3,4 from test3
779e00f 1st commit t1,t2t3 cotent1,12,123
[root@ip-172-31-39-124 git2]# git rebase -i
There is no tracking information for the current branch.
Please specify which branch you want to rebase against.
See git-rebase(1) for details.
```

```
git rebase '<branch>'
```

If you wish to set tracking information for this branch you can do so with:

```
git branch --set-upstream-to=<remote>/<branch> master
```

```
[root@ip-172-31-39-124 git2]# git rebase -i 7d4a83a
Successfully rebased and updated refs/heads/master.
[root@ip-172-31-39-124 git2]# git log --oneline
7d4a83a (HEAD -> master) 5th commit from master test3 with 5,6
ddccede Merge branch 'dev' changed the content of t2 file eventhough having 2 commits from dev and made a 2nd commit in master to perfrom a git conflict
9555f23 2nd commit from master test2 content 3
36f34a0 2nd commit from dev content 12,3,4 from test3
779e00f 1st commit t1,t2t3 cotent1,12,123
```

Git command to view recent 2 commits `git log -n`

Assignment 5

Jumbling git log with git rebase -I HEAD~[no of commits]

A dialogue box will open in that move the commits as you need,

```
[root@ip-172-31-39-124 git2]# git rebase -i HEAD~3
Successfully rebased and updated refs/heads/master.
[root@ip-172-31-39-124 git2]# git log --oneline
fd517c0 (HEAD -> master) 5th commit from master test3 with 5,6
d81179f 2nd commit from dev content 12,3,4 from test3
9555f23 2nd commit from master test2 content 3
779e00f 1st commit t1,t2t3 cotent1,12,123
[root@ip-172-31-39-124 git2]# git rebase -i HEAD~3
Successfully rebased and updated refs/heads/master.
[root@ip-172-31-39-124 git2]# git log --oneline
b4b6cb8 (HEAD -> master) 2nd commit from master test2 content 3
0c1fefaf 5th commit from master test3 with 5,6
2a5d37a 2nd commit from dev content 12,3,4 from test3
779e00f 1st commit t1,t2t3 cotent1,12,123
[root@ip-172-31-39-124 git2]#
```

```

pick 9555f23 2nd commit from master test2 content 3
pick d81179f 2nd commit from dev content 12,3,4 from test3
pick fd517c0 5th commit from master test3 with 5,6
pick 9555f23 2nd commit from master test2 content 3
# Rebase 779e00f..fd517c0 onto 779e00f (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
# .       message (or the oneline, if no original merge commit was
# .       specified); use -c <commit> to reword the commit message
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#

```

3rd scenario git rebase

- Make 2 commits in master [1,2]
- Make 3 commits in dev [1,2,3]
- Checkout to master and make one commit in master, [1,2,] and make a commit in new file [4]
- So dev 3 commit ,master 3 commit
- Stand in feature branch and give git rebase master
- So in dev it will be 1,2,3 and 4 this 4 came from master to feature

- In master it will be 1,2,4 and 3 is missing in master
- Checkout to master and give git merge then in master ,3 will came from feature to master
- By using git rebase the content will be copied from **master to feature** after tht we can give git rebase to copy it from **feature to master**.

```
[root@ip-172-31-39-124 git4]# git rebase master
Successfully rebased and updated refs/heads/dev.
[root@ip-172-31-39-124 git4]# cat test
1
2
3
[root@ip-172-31-39-124 git4]# ll
total 8
-rw-r--r-- 1 root root 6 Jul  4 15:12 test
-rw-r--r-- 1 root root 2 Jul  4 15:12 test1
[root@ip-172-31-39-124 git4]# git status
On branch dev
nothing to commit, working tree clean
[root@ip-172-31-39-124 git4]# git log --oneline
5fd4295 (HEAD -> dev) 3rd commit from dev 3
172376e (master) 4th commit from master@test1 4
101cba6 2nd commit c=2
1d2458b 1st commit c=1
[root@ip-172-31-39-124 git4]# git checkout master
Switched to branch 'master'
[root@ip-172-31-39-124 git4]# git log --oneline
172376e (HEAD -> master) 4th commit from master@test1 4
101cba6 2nd commit c=2
1d2458b 1st commit c=1
[root@ip-172-31-39-124 git4]# ll
total 8
-rw-r--r-- 1 root root 4 Jul  4 15:13 test
```

Git cherry-pick

- Make 2 commits from master ,switch to dev
- Make 3 and 4 th commit thought that you hhave done it by mistake
- Switch to master ,git cherry-pick [3rd commit id]

```

[root@ip-172-31-39-124 git5]# git commit -am "3rd commit from dev c=3"
[dev e0d0ed7] 3rd commit from dev c=3
1 file changed, 1 insertion(+)
[root@ip-172-31-39-124 git5]# vi test
[root@ip-172-31-39-124 git5]# git commit -am "4th commit from dev c=4"
[dev 7cdfef8] 4th commit from dev c=4
1 file changed, 1 insertion(+)
[root@ip-172-31-39-124 git5]# git log --oneline
7cdfef8 (HEAD -> dev) 4th commit from dev c=4
e0d0ed7 3rd commit from dev c=3
cbe1b07 (master) 2nd commit from master 1,2
0399a22 1st commit
[root@ip-172-31-39-124 git5]# git checkout master
Switched to branch 'master'
[root@ip-172-31-39-124 git5]# git cherry-pick e0d0ed7
[master 25d0f50] 3rd commit from dev c=3
Date: Sun Jul 4 15:47:00 2021 +0000
1 file changed, 1 insertion(+)
[root@ip-172-31-39-124 git5]# git log --oneline
25d0f50 (HEAD -> master) 3rd commit from dev c=3
cbe1b07 2nd commit from master 1,2
0399a22 1st commit
[root@ip-172-31-39-124 git5]# cat test
1
2
3

```

now you've stole the 3rd content with commit id

- Git pull - to fetch and merge the changes from github to local repository
- Git fetch - If you only want to see all of the current branches and changes in your remote repository, **Git fetch** can get you all of the information you need without actually making any local changes to your work. it will download the contents from remote to local without making any changes
- Git merge - will merge the changes that's downloaded from remote repo

Branching strategy

- 1] Tickets will come from ops team ,after capturing the branch information
- 2] Pull the code to local repository
- 3] Take a feature branch and make required changes
- 4] Code will be tagged and pushed to my github remote repository
- 5] A pull request will be created for the reviewer ,reviewer[teach lad,project manger] after reviewing and verifying the code like a maker-checker
- 6] Master merge will happen